

Acceso a Datos _ Sesión 22-09-2025

Marco normativo y contexto

- El título oficial de Técnico Superior en Desarrollo de Aplicaciones Multiplataforma.
- Módulo “Acceso a datos” como parte del 2º curso.
- Duración del módulo / horas oficiales: aproximadamente 80 horas

Objetivos del módulo

Al finalizar el módulo, el alumnado deberá ser capaz de:

1. Conocer y manejar distintos tipos de almacenamiento de datos: ficheros, bases de datos relacionales, orientadas a objetos, no relacionales (documentales).
2. Diseñar el esquema lógico y físico de bases de datos, aplicando normalización y garantizando integridad, consistencia, seguridad y accesibilidad de los datos.
3. Establecer conexiones desde aplicaciones a bases de datos, ejecutar consultas, inserciones, modificaciones y eliminaciones de datos.
4. Utilizar procedimientos almacenados, triggers, vistas, y otras herramientas propias de los SGBD.
5. Trabajar con herramientas ORM (mapeo objeto-relacional) para facilitar el acceso a datos desde lenguajes orientados a objetos.
6. Manejar formatos de intercambio de datos (por ejemplo JSON, XML), serialización/deserialización, transformación.
7. Gestionar transacciones, concurrencia, bloqueo, y entender el rendimiento del acceso a datos.
8. Asegurar la persistencia y protección de los datos (backup, recuperación, gestión de errores).
9. Integrar los componentes de acceso a datos en aplicaciones multiplataforma cumpliendo especificaciones de calidad, eficiencia y usabilidad.

Contenidos temáticos sugeridos

Unidad	Contenidos
Unidad 1: Introducción al acceso a datos	• Conceptos básicos: datos, información, almacenamiento, persistencia. • Tipos de almacenamiento: ficheros vs. bases de datos. • Modelos de datos: relacional, orientado a objetos, documental. • Ventajas e inconvenientes de cada modelo. • Requisitos no funcionales: seguridad, integridad, concurrencia, rendimiento.
Unidad 2: Ficheros y formatos de intercambio	• Manejo de ficheros: tipos (texto, binario), operaciones de lectura/escritura. • Acceso secuencial y aleatorio. • Serialización y deserialización de objetos. • Formatos comunes de intercambio: XML, JSON, CSV. • Parsers, validación, binding. • Manejo de errores/excepciones relacionados con ficheros.
Unidad 3: Bases de datos relacionales	• Fundamentos: tablas, claves primarias y foráneas, esquemas. • Diseño lógico: normalización (1NF, 2NF, 3NF, etc.). • Diseño físico: índices, particionamiento, vistas. • Lenguaje SQL: DDL, DML, DCL, consultas; joins; subconsultas; agregados; funciones.
Unidad 4: Procedimientos almacenados, triggers e integridad	• Definición y uso de procedimientos almacenados. • Triggers (disparadores). • Vistas y sinónimos. • Integridad referencial y otras restricciones. • Transacciones: definición, propiedades (ACID), aislamiento, bloqueo.
Unidad 5: Bases de datos no relacionales (documentales u otros)	• Características de bases de datos no relacionales/documentales. • Modelos de documentos (ej. JSON, BSON). • Operaciones CRUD en bases de datos documentales. • Indexación, consultas, agregaciones en bases no relacionales.

Unidad 6: ORM y acceso desde lenguajes de programación

- Principio del mapeo objeto-relacional.
- Herramientas ORM populares (por ejemplo Hibernate, Entity Framework, etc.).
- Configuración y uso de ORM: entidades/clases persistentes, relaciones, lazy loading, transacciones.
- Comparación entre ORM vs SQL puro en determinados escenarios.

Unidad 7: Conectividad y componentes de acceso

- Drivers y conectores (JDBC, ODBC, otros).
- Pooling de conexiones.
- Componentes que abstraen el acceso a datos.
- Seguridad: autenticación, autorización, cifrado en tránsito.
- Pruebas de acceso a datos (unitarias, integración).

Unidad 8: Rendimiento, optimización y buenas prácticas

- Optimización de consultas y uso de índices.
- Análisis de planes de ejecución.
- Tuning de bases de datos.
- Cacheado.
- Gestión de concurrencia y conflictos.
- Escalabilidad: particionamiento, replicación, clustering.

Unidad 9: Integración y aplicaciones multiplataforma

- Cómo integrar el módulo de acceso a datos en aplicaciones multiplataforma (web, móvil, escritorio).
- Acceso remoto a bases de datos: servicios web, APIs.
- Formatos de respuesta, JSON / XML / servicios REST, GraphQL.
- Consideraciones de seguridad, latencia, conexiones móviles.
- Persistencia en dispositivos móviles y sincronización.

Unidad 10: Proyecto práctico / casos de estudio

- Análisis de requerimientos de acceso a datos.
- Diseño y construcción de una base de datos.
- Implementación del acceso desde una aplicación real o simulada.
- Integración de ficheros / intercambio de datos.
- Optimización, seguridad.
- Documentación y pruebas.

Tema 1: Introducción al Acceso a Datos

1. Conceptos básicos

1.1. Dato e información

- **Dato:** representación mínima de un hecho, sin contexto.
Ejemplo: 25
- **Información:** dato interpretado en un contexto.
Ejemplo: 25 años (edad de una persona).

1.2. Persistencia

- **Persistencia:** propiedad que permite que los datos sobrevivan al fin de un programa.
- **Memoria volátil:** RAM (se borra al apagar).
- **Memoria persistente:** disco, SSD, base de datos.

Ejemplo en Java:

```
// Guardar un dato en memoria RAM
int edad = 25;
```

```
// Guardar un dato en un fichero (persistente)
import java.io.*;
```

```
public class GuardarDato {
    public static void main(String[] args) throws IOException {
        FileWriter fw = new FileWriter("edad.txt");
        fw.write("25");
        fw.close();
    }
}
```

2. Tipos de almacenamiento

2.1. Ficheros

- **Texto plano:** CSV, TXT → legibles por humanos.
- **Binarios:** estructuras codificadas → más rápidos, menos legibles.
- **Acceso:**
 - **Secuencial:** leer línea por línea.
 - **Aleatorio:** saltar a una posición del archivo (clase `RandomAccessFile` en Java).

2.2. Bases de datos

- **Relacionales (SQL)**
 - Modelo tabular.
 - Lenguaje: SQL.
 - Ejemplo: PostgreSQL, MySQL.
- **No relacionales (NoSQL)**
 - Flexibles, sin esquema fijo.
 - Ejemplo: MongoDB (documental), Redis (clave-valor), Neo4j (grafos).
- **Orientadas a objetos (OODBMS)**
 - Guardan objetos completos.
 - Ejemplo: ObjectDB.

3. Modelos de datos

3.1. Relacional

```
CREATE TABLE Alumno (  
  id SERIAL PRIMARY KEY,  
  nombre VARCHAR(100) NOT NULL,  
  edad INT CHECK (edad >= 0)  
);
```

```
INSERT INTO Alumno (nombre, edad) VALUES ('Laura', 22);  
SELECT * FROM Alumno;
```

3.2. Documental (MongoDB)

```
{  
  "id": 1,  
  "nombre": "Laura",  
  "edad": 22,  
  "asignaturas": ["AD", "PSP", "PMDM"]  
}
```

4. Requisitos del acceso a datos

- **Integridad:** claves, restricciones, normalización.
- **Concurrencia:** bloqueo, transacciones ACID.
- **Seguridad:** roles, permisos, cifrado.
- **Rendimiento:** índices, optimización de consultas, cacheado.

Ejemplo SQL de permisos:

```
GRANT SELECT, INSERT ON Alumno TO profesor;  
REVOKE DELETE ON Alumno FROM profesor;
```

5. Arquitectura de acceso a datos

- **Patrón DAO (Data Access Object):** capa que encapsula el acceso a datos.
- **ORM (Object Relational Mapping):** Hibernate, JPA, Entity Framework.
- **Conexión con JDBC (Java Database Connectivity):**

```
import java.sql.*;

public class ConexionBD {
    public static void main(String[] args) throws Exception {
        Connection conn = DriverManager.getConnection(
            "jdbc:postgresql://localhost:5432/academia", "postgres", "1234");

        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery("SELECT * FROM Alumno");

        while (rs.next()) {
            System.out.println(rs.getInt("id") + " - " + rs.getString("nombre"));
        }

        conn.close();
    }
}
```

Ejercicios prácticos Tema 1

Ejercicio 1 – Persistencia en ficheros

1. Crea un programa en Java que guarde en un fichero `alumnos.txt` los nombres de tres alumnos.
2. Haz otro programa que lea el fichero y muestre los nombres en consola.

Ejercicio 2 – SQL básico

1. Crea una tabla `Asignatura` con los campos: `id`, `nombre`, `horas`.
2. Inserta al menos 3 registros.
3. Haz una consulta que muestre todas las asignaturas con más de 3 horas.

Ejercicio 3 – Modelo documental

1. Representa el siguiente dato en formato JSON:
 - Un alumno con nombre *“Carlos”*, edad *21*, y asignaturas: *“AD”*, *“PSP”*.
2. Explica qué diferencia hay entre guardar este JSON en un fichero y almacenarlo en MongoDB.

Ejercicio 4 – Conexión a base de datos

1. Conéctate desde Java a una base de datos PostgreSQL llamada `academia`.
2. Muestra en consola todos los alumnos de la tabla `Alumno`.

Simulación de clase

Ejercicio 1

Por consola, te pide que introduzcas una ciudad

La introduces, y se pide otra...hasta que le digas que no quieres más.

Por ejemplo, pulsando q

cómo se guardan las ciudades?array, arraylist, set, diccionario....

si alguien repite la ciudad no error, pero NO se guarda dos veces *

data NO es persistente.

Para convertirlo en persistente, almacena en un fichero csv

Escenario 1 - java

Escenario 2 - php

Escenario 3 - python

Solución

Estructuras de datos – Características

Estructura	Características	Ventajas	Inconvenientes	Ejemplo uso
Array	Tamaño fijo, acceso por índice, elementos ordenados.	Muy eficiente en acceso por posición.	No admite inserciones dinámicas (tamaño fijo en Java y PHP).	Almacenar 10 notas fijas.
ArrayList / List	Tamaño dinámico, permite duplicados, mantiene orden.	Fácil inserción, búsqueda por índice.	Inserciones/borrados internos menos eficientes.	Lista de alumnos.
Set	Conjunto no ordenado, no admite duplicados .	Garantiza unicidad automáticamente.	No acceso por índice.	Conjunto de ciudades únicas.
Map / Diccionario	Pares clave → valor, no duplicados en clave.	Búsqueda muy rápida por clave.	Complejidad mayor.	Agenda telefónica.

En este ejercicio la estructura más adecuada es un **Set**, porque queremos **evitar duplicados** de ciudades automáticamente.

Ejercicio propuesto

Programa que:

1. Pide ciudades por consola, hasta que el usuario escriba **q**.
 2. No permite ciudades repetidas.
 3. Al final, guarda el resultado en un fichero CSV (una ciudad por línea).
-

Escenario 1 – Java

```
import java.io.FileWriter;
import java.io.IOException;
import java.util.HashSet;
import java.util.Scanner;
import java.util.Set;

public class Ciudades {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Set<String> ciudades = new HashSet<>();
        String input;

        while (true) {
            System.out.print("Introduce una ciudad (q para salir): ");
            input = sc.nextLine().trim();

            if (input.equalsIgnoreCase("q")) break;

            if (ciudades.add(input)) {
                System.out.println("Ciudad añadida.");
            } else {
                System.out.println("Ya existe esa ciudad, no se añade.");
            }
        }

        // Guardar en CSV
    }
}
```

```

        try (FileWriter fw = new FileWriter("ciudades.csv")) {
            for (String ciudad : ciudades) {
                fw.write(ciudad + "\n");
            }
            System.out.println("Ciudades guardadas en ciudades.csv");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

Escenario 2 – PHP

```

<?php
$ciudades = [];

while (true) {
    echo "Introduce una ciudad (q para salir): ";
    $input = trim(fgets(STDIN));

    if (strtolower($input) === "q") break;

    if (!in_array($input, $ciudades)) {
        $ciudades[] = $input;
        echo "Ciudad añadida.\n";
    } else {
        echo "Ya existe esa ciudad, no se añade.\n";
    }
}

// Guardar en CSV
$file = fopen("ciudades.csv", "w");
foreach ($ciudades as $ciudad) {
    fputcsv($file, [$ciudad]);
}
fclose($file);

echo "Ciudades guardadas en ciudades.csv\n";
?>

```

Escenario 3 – Python

```
import csv
```

```
ciudades = set()
```

```
while True:
```

```
    ciudad = input("Introduce una ciudad (q para salir: ").strip()
```

```
    if ciudad.lower() == "q":
```

```
        break
```

```
    if ciudad not in ciudades:
```

```
        ciudades.add(ciudad)
```

```
        print("Ciudad añadida.")
```

```
    else:
```

```
        print("Ya existe esa ciudad, no se añade.")
```

```
# Guardar en CSV
```

```
with open("ciudades.csv", "w", newline="") as f:
```

```
    writer = csv.writer(f)
```

```
    for ciudad in ciudades:
```

```
        writer.writerow([ciudad])
```

```
print("Ciudades guardadas en ciudades.csv")
```

Comparativa entre los tres escenarios

- **Java:** Uso de `Set<String>` (`HashSet`) para evitar duplicados. Es más verboso, pero fuerte tipado y control.
- **PHP:** No tiene `Set` nativo → se usa un **array** y comprobación manual con `in_array()`.
- **Python:** Tiene `set` nativo, simple y directo.

Ejercicio extendido: Ciudades sin duplicados

Planteamiento base

1. El programa pide ciudades por consola hasta que el usuario escriba `q`.
2. Una ciudad repetida no se añade.
3. Al final, se guardan en un fichero `ciudades.csv`.

Parte A – Implementación con diferentes estructuras de datos

Escenario 1: Usando **Set** (sin duplicados automáticamente)

Código ya visto (más óptimo en Java y Python).

Escenario 2: Usando **ArrayList** (Java), **Array** (PHP), **list** (Python)

Aquí **debemos comprobar duplicados manualmente** antes de insertar.

Java – con **ArrayList**

```
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.Scanner;

public class CiudadesArrayList {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<String> ciudades = new ArrayList<>();
        String input;

        while (true) {
            System.out.print("Introduce una ciudad (q para salir): ");
            input = sc.nextLine().trim();

            if (input.equalsIgnoreCase("q")) break;

            if (!ciudades.contains(input)) {
                ciudades.add(input);
                System.out.println("Ciudad añadida.");
            } else {
                System.out.println("Ya existe esa ciudad, no se añade.");
            }
        }

        try (FileWriter fw = new FileWriter("ciudades.csv")) {
            for (String ciudad : ciudades) {
                fw.write(ciudad + "\n");
            }
            System.out.println("Ciudades guardadas en ciudades.csv");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

```
}
```

PHP – con **array**

```
<?php
$ciudades = [];

while (true) {
    echo "Introduce una ciudad (q para salir): ";
    $input = trim(fgets(STDIN));

    if (strtolower($input) === "q") break;

    if (!in_array($input, $ciudades)) {
        $ciudades[] = $input;
        echo "Ciudad añadida.\n";
    } else {
        echo "Ya existe esa ciudad, no se añade.\n";
    }
}

$file = fopen("ciudades.csv", "w");
foreach ($ciudades as $ciudad) {
    fputcsv($file, [$ciudad]);
}
fclose($file);

echo "Ciudades guardadas en ciudades.csv\n";
?>
```

Python – con **list**

```
import csv

ciudades = []

while True:
    ciudad = input("Introduce una ciudad (q para salir: ").strip()
    if ciudad.lower() == "q":
        break

    if ciudad not in ciudades:
        ciudades.append(ciudad)
        print("Ciudad añadida.")
    else:
```

```
print("Ya existe esa ciudad, no se añade.")

with open("ciudades.csv", "w", newline="") as f:
    writer = csv.writer(f)
    for ciudad in ciudades:
        writer.writerow([ciudad])

print("Ciudades guardadas en ciudades.csv")
```

Escenario 3: Usando Map/Diccionario

Aquí almacenamos la ciudad como **clave** y un valor asociado (ejemplo: número de veces introducida).

Esto permite explicar **estructuras clave-valor**.

Java – HashMap

```
import java.io.FileWriter;
import java.io.IOException;
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class CiudadesMapa {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Map<String, Integer> ciudades = new HashMap<>();
        String input;

        while (true) {
            System.out.print("Introduce una ciudad (q para salir): ");
            input = sc.nextLine().trim();

            if (input.equalsIgnoreCase("q")) break;

            ciudades.put(input, ciudades.getOrDefault(input, 0) + 1);
            System.out.println("Ciudad añadida (veces introducida: " + ciudades.get(input) + ")");
        }

        try (FileWriter fw = new FileWriter("ciudades.csv")) {
            for (String ciudad : ciudades.keySet()) {
                fw.write(ciudad + ";" + ciudades.get(ciudad) + "\n");
            }
            System.out.println("Ciudades guardadas en ciudades.csv");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

PHP – array asociativo

```
<?php
$ciudades = [];
```

```
while (true) {
```

```

echo "Introduce una ciudad (q para salir): ";
$input = trim(fgets(STDIN));

if (strtolower($input) === "q") break;

if (isset($ciudades[$input])) {
    $ciudades[$input]++;
} else {
    $ciudades[$input] = 1;
}
echo "Ciudad añadida (veces introducida: {$ciudades[$input]})\n";
}

$file = fopen("ciudades.csv", "w");
foreach ($ciudades as $ciudad => $veces) {
    fputcsv($file, [$ciudad, $veces]);
}
fclose($file);

echo "Ciudades guardadas en ciudades.csv\n";
?>

```

Python – dict

```

import csv

ciudades = {}

while True:
    ciudad = input("Introduce una ciudad (q para salir: ").strip()
    if ciudad.lower() == "q":
        break

    ciudades[ciudad] = ciudades.get(ciudad, 0) + 1
    print(f"Ciudad añadida (veces introducida: {ciudades[ciudad]})")

with open("ciudades.csv", "w", newline="") as f:
    writer = csv.writer(f)
    for ciudad, veces in ciudades.items():
        writer.writerow([ciudad, veces])

print("Ciudades guardadas en ciudades.csv")

```